

HONEYWELL EMBEDDED ENGINEERING TEAMS — PA · IA · BA

AI Champions Programme

Module 07

MCP-Enabled Agentic Embedded Engineering and Reusable Skills

Connecting agents to your real engineering tools — repositories, build, test, static analysis, issue tracking — under governed permissions, then packaging what works as a reusable skill.



DURATION

2 Hours · Module 7 of 12



FORMAT

Guided Build + Skill Packaging Lab



DELIVERED FOR

Honeywell Embedded Engineering

How We'll Spend These Two Hours

Six connected moves — from MCP architecture to a packaged, versioned skill your team can reuse.



01

MCP Architecture & What Agents Can Reach

20 min

Servers, tools & resources — the engineering surface an agent can be connected to.



02

Permissions, Approvals & Boundaries

20 min

Tool selection, context/state boundaries, and what always needs a human sign-off.



03

Multi-Agent Patterns

15 min

Planner, architect, developer, tester, reviewer, release checker — as roles, not job titles.



04

Build an MCP-Enabled Workflow

35 min

Repository analysis → spec validation → build/test → PR evidence, wired end to end.



05

Package a Reusable Skill

20 min

Turn today's workflow into a versioned, owned asset the whole team can reuse.



06

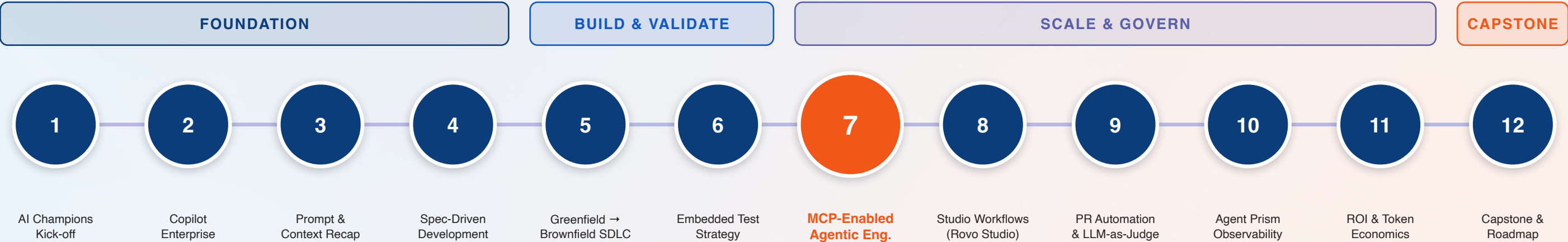
Failure Handling & Security Review

10 min

What happens when a tool call fails, and who's accountable when it does.

Module 07 in the 12-Module Journey

Where the engineering foundation from Modules 01–06 becomes an agent's operating surface.



Why this matters: Module 08's studio workflows are MCP-style automation authored by business and program roles. Module 09's PR quality gates and Module 10's Agent Prism both observe the exact tool calls this module teaches you to wire up.

MCP Architecture: Servers, Tools & Resources

One protocol, a governed set of connections — the vocabulary for everything else today.



Agent / Client

The Copilot agent driving the workflow

Sends requests through MCP, never touches your systems directly — every action goes through a server.



MCP Server

A governed gateway to one engineering system

Exposes a bounded set of tools & resources for one system — your Git host, build system, or issue tracker.



Tools

Actions the agent can invoke

Run a build, execute a test suite, create a draft PR — each one explicit, permissioned, and logged.



Resources

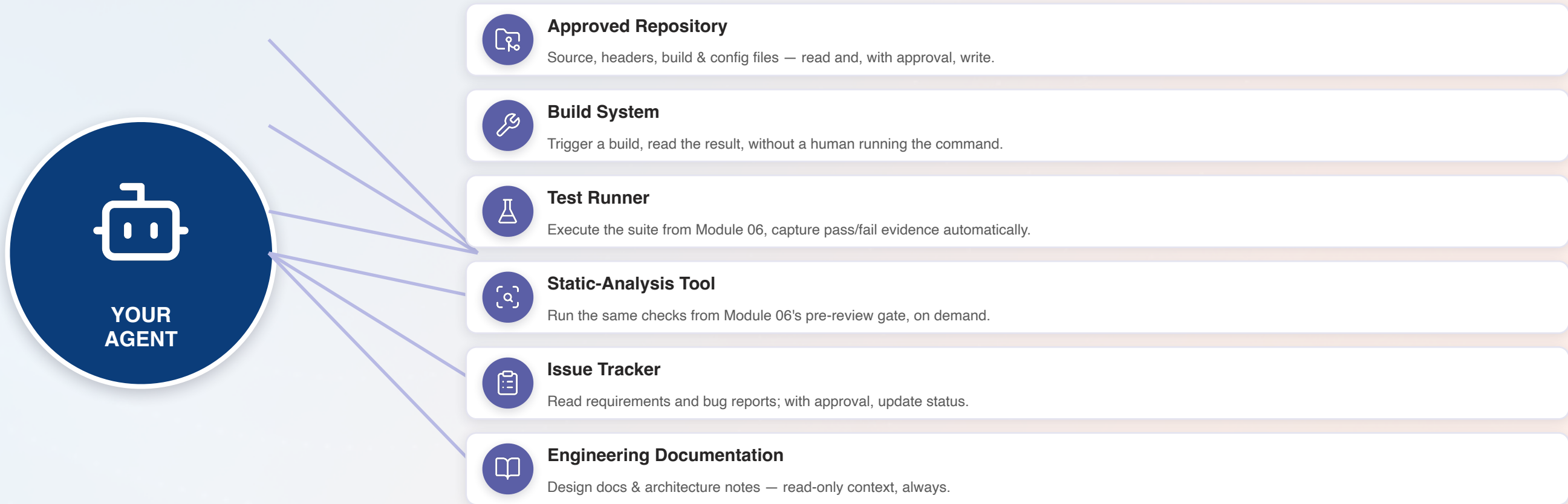
Data the agent can read

Repository files, design documents, issue history — available for context, not for silent modification.

The governing idea: an agent's capability is exactly the set of tools it's been granted — nothing implicit, nothing assumed.

What Your Agent Can Reach

Six engineering systems, each behind its own governed MCP server.



What the Calibration Survey Told Us About Agentic Workflows

Real answers from 19 Honeywell engineers — this is the module they asked for first.

Top Training Ask, Revisited

13

of 19

Back on Slide 6 of Module 01, “learn MCP and agentic workflows” topped every other training ask by a wide margin. Today is that module.

Independently ranked, “agentic workflows” (9 of 19) and “MCP & tool integration” (7 of 19) both placed in the top half of desired capabilities.

Only 2 of 19 said “not sure” when asked to imagine an MCP-connected workflow — 17 of 19 already described one, unprompted.

The Workflow This Cohort Already Imagined

“I’d first use it for issue-to-fix automation — gather requirements, locate code, build, run tests, and create a draft PR. This saves engineering time while keeping humans in control of design and review.”

— Paraphrased from multiple independent survey responses



Issue → fix → validated PR

The dominant pattern across responses



Static analysis → auto-filed issues

A close second, from PA & IA engineers



“Humans stay in control of design & review”

Named explicitly, unprompted, by several respondents

Today's build: is almost exactly the workflow this cohort described on its own — we're formalizing an instinct, not introducing a foreign concept.

Tool Selection, Permissions & Approvals

Every tool call sits on a spectrum — from always-allowed to always-approved.

READ-ONLY — GENERALLY AUTONOMOUS

-  Reading repository files, headers & config
-  Running static analysis & reading results
-  Executing the existing test suite
-  Reading issues, docs & design notes

WRITE / EXECUTE — ALWAYS APPROVAL-GATED

-  Committing or pushing any code change
-  Opening, updating, or merging a pull request
-  Triggering a production build or release pipeline
-  Closing or reassigning an issue

The default posture: grant read access broadly, grant write/execute access narrowly, and never assume yesterday's approval covers today's task.

Context Boundaries, Failure Handling & Security

What happens when a multi-step workflow goes wrong — planned for, not improvised.



Context & State Boundaries

Each workflow step gets exactly the context it needs — Module 03's discipline, applied to multi-step agent runs.



Tool Call Failure

A failed build or timed-out test halts the workflow and reports why — it never silently continues on bad data.



Escalation Path

Ambiguous results or repeated failures route to a human, with the full trace attached.



Security Review

Every MCP server connection is reviewed like any other production integration — least privilege, audited access.

Rule of thumb: if a workflow can't explain why it stopped, it shouldn't have been allowed to keep going.

Multi-Agent Patterns: Six Engineering Roles

Not job titles — functions an agent (or a human) can perform within a governed workflow.



Planner

Breaks a request into ordered, verifiable steps — Module 04's task decomposition, agent-side.



Architect

Checks a proposed change against interfaces & HAL boundaries before implementation starts.



Developer

Implements tasks against the spec, the same discipline as Module 02's Agent Mode.



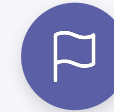
Tester

Runs the Module 06 test strategy and reports evidence, not just a pass/fail summary.



Reviewer

Checks the diff against spec, standards & test evidence — Module 09's LLM-as-Judge role.



Release Checker

Confirms every release-readiness item from Module 05's checklist before sign-off.

One agent or six: a single agent can play multiple roles sequentially, or you can wire distinct agents per role — the roles themselves don't change either way.

Building an MCP Workflow: Issue to Validated PR

The exact pattern this cohort described unprompted — now wired end to end.



Where the human stays in control: the PR is a draft. Merge is a human decision, every time — exactly what the cohort's own answers described.

Packaging Reusable Skills

Today's workflow, turned into an asset the whole team can pick up — not just today's engineer.



Design Explorer

Walks a new team member through an unfamiliar module's architecture on request.



Test Enhancement Assistant

Suggests boundary & negative cases missing from an existing test file.



PR Reviewer

Runs the Module 09 quality checklist automatically on every incoming PR.



RCA Assistant

Given a regression, traces it back through call hierarchy & recent changes.



Versioning & Ownership — Not Optional

Every packaged skill carries a version number, a named owner, and a changelog — the same discipline Module 04 applies to specifications. An unowned skill is one nobody will trust to update.

Hands-On Lab: Build the Workflow, Package the Skill

Two connected exercises — wire the pipeline, then make it reusable for everyone else.

PART 1 BUILD

- Wire an MCP-enabled workflow: repository analysis → spec validation → build/test → PR evidence.
- Use a real issue from the Module 05/06 artefacts as the starting point.

PART 2 PACKAGE

- Choose one skill — design explainer, test assistant, PR reviewer, or RCA assistant — and package it.
- Assign a version, an owner, and write the one-paragraph description someone else would need.



Test a Failure

Deliberately fail one tool call — confirm the workflow halts and explains why.



Instructor Review

Facilitators check permission scoping against the read/write split from Slide 7.



Carry Forward

Both the workflow and the packaged skill feed Module 08's studio-workflow lab.

Anti-Patterns: How Agentic Workflows Go Wrong

Five habits that turn a governed workflow into an ungoverned one.

DO THIS

-  Grant exactly the tools a workflow needs, nothing more
-  Keep merge/release actions behind a human approval gate
-  Halt and report clearly when a tool call fails
-  Package a skill with a named owner and a version
-  Review MCP server connections like any production integration

NOT THAT

-  Grant broad, standing access “to save time re-approving”
-  Let an agent merge its own PR because it “usually gets it right”
-  Retry silently or continue on partial, unverified results
-  Leave a useful prompt in someone's chat history, unshared
-  Treat a server connection as a one-time setup, never revisited

Module 07 Outcomes & What's Next

Everything below should be true before program and product roles author workflows in Module 08.



Can explain MCP architecture: agents, servers, tools & resources



Understand the read-only vs. approval-gated permission split



Know all six multi-agent roles: planner, architect, developer, tester, reviewer, release checker



Packaged a reusable skill with a version, an owner, and a changelog



Know what your agent can reach: repo, build, test, static analysis, issues, docs



Can explain context boundaries, failure handling & escalation paths



Built a real MCP workflow: repository analysis → spec validation → build/test → PR evidence



Know the five anti-patterns that turn governed workflows into ungoverned ones



NEXT — MODULE 08: STUDIO-STYLE WORKFLOWS FOR BUSINESS AND ENGINEERING COLLABORATION

2 hours · Program & product roles author governed, agent-assisted workflows in Atlassian Rovo Studio — grounded on Jira & Confluence, without bypassing engineering governance.

Questions & Discussion

Module 07 — MCP-Enabled Agentic Embedded Engineering and Reusable Skills